

Міністерство освіти і науки України
Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Звіт
з лабораторної роботи No 8
з дисципліни
«Мова програмування Java»

Виконала:
студентка групи ЗПІ-зп41
Ірина Левківська

Перевірив:
Орленко С.П.

Київ 2025

Поставка завдання (Варіант 2)

Напишіть консольний додаток для обчислення суми всіх елементів масиву (з 1_000_000 цілих елементів, значення яких генеруються випадковим чином в діапазоні від 0 до 100), використовуючи фреймворк ForkJoin. Для цього:

- створіть та ініціалізуйте масив розмірністю 1_000_000 елементів;
- опишіть рекурсивну задачу для поділу масиву на дві частини і виклик для кожної частини рекурсивного завдання для подальшого розподілу;
- продовжуйте розподіл масиву, поки в підмасиві не залишиться менше 20 елементів, і тоді обчисліть суму цих елементів.

Вирішення завдання

Для виконання завдання я використала архітектуру ForkJoin, яка входить до пакету `java.util.concurrent`. Основна ідея полягає в тому, щоб не змушувати один потік рахувати мільйон чисел, а розбити цей масив на багато дрібних сегментів.

Ключові моменти реалізації:

- Рекурсивна задача: Я створила клас `SumTask`, який успадковує `RecursiveTask<Long>`. Оскільки результатом обчислення є число (сума), використовується саме `RecursiveTask`, а не `RecursiveAction`.
- Умова виходу (Threshold): Згідно з умовою, коли кількість елементів у підмасиві стає меншою або рівною 20, рекурсія зупиняється, і програма переходить до прямого підрахунку суми. Це дозволяє уникнути зайвих витрат ресурсів на створення надто дрібних об'єктів задач.
- Механізм Fork та Join: * `fork()` відправляє ліву підзадачу в пул для виконання іншим вільним потоком. `compute()` для правої частини виконується в поточному потоці, щоб зекономити ресурси. `join()` очікує завершення фонових обчислень та повертає результат.

```
// Клас рекурсивної задачі
static class SumTask extends RecursiveTask<Long> {
    private static final int THRESHOLD = 20;
    private int[] array;
    private int start;
    private int end;

    public SumTask(int[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }
}
```

```

@Override
protected Long compute() {
    // Якщо елементів менше порогу, рахуємо суму прямо тут
    if (end - start <= THRESHOLD) {
        long sum = 0;
        for (int i = start; i < end; i++) {
            sum += array[i];
        }
        return sum;
    } else {
        // Інакше - ділимо задачу навпіл
        int middle = (start + end) / 2;
        SumTask leftTask = new SumTask(array, start, middle);
        SumTask rightTask = new SumTask(array, middle, end);

        leftTask.fork();
        long rightResult = rightTask.compute();
        // Чекаємо завершення лівої частини та додаємо результати
        long leftResult = leftTask.join();

        return leftResult + rightResult;
    }
}
}

```

```

public static void main(String[] args) {
    int size = 1_000_000;
    int[] array = new int[size];
    Random random = new Random();

    // Ініціалізація масиву випадковими числами [0, 100]
    for (int i = 0; i < size; i++) {
        array[i] = random.nextInt(101);
    }

    // Створення пулу потоків
    ForkJoinPool pool = new ForkJoinPool();

    // Вимірювання часу та обчислення суми
    long startTime = System.currentTimeMillis();
    long totalSum = pool.invoke(new SumTask(array, 0, array.length));
    long endTime = System.currentTimeMillis();

    System.out.println("Сума всіх елементів масиву: " + totalSum);
    System.out.println("Час виконання (ForkJoin): " + (endTime - startTime) + " мс");

    // Перевірка (звичайний цикл для контролю)
    long checkSum = 0;
    for (int val : array) checkSum += val;
    System.out.println("Контрольна сума: " + checkSum);
}
}

```

Скріншоти виконання коду

```
Сума всіх елементів масиву: 49949600  
Час виконання (ForkJoin): 41 мс  
Контрольна сума: 49949600
```

Висновки

Реалізація цього завдання продемонструвала мені ефективність фреймворку ForkJoin для задач, які можна розділити на незалежні частини. Я навчилася правильно визначати поріг рекурсії (THRESHOLD) та використовувати методи `fork()` і `join()` для синхронізації паралельних потоків.